

Silent Wrong Answers

One theme joins all three things you're presenting: a check that cannot fail is worse than no check at all. The peer review said it about hypotheses; both pull requests found it in code that reported success while doing nothing.

Runs ~8 min **Covers** artifact 920a5e75 + PR #22 + PR #23 **Both PRs** open, from your fork

Open with the through-line

“Three things this week came down to the same idea. A test that always passes tells you nothing. An installer that exits zero without installing anything tells you nothing. And a hypothesis that can't be wrong isn't a hypothesis.”

Part 1 – the peer review, and what changed because of it

Artifact: *What Would Prove It Wrong*. Three minutes.

0:00–1:00

The criticism that landed

- “You're justifying your hypotheses with papers.” That was fair about the order I worked in.
- I fixed six hypotheses first, then sorted 58 papers into them. Every column in my spreadsheet asked “does this paper support something I already believe?”
- There was no column where a paper could say “here's something you haven't thought of.” So that answer could never appear.

1:00-2:00

What replaced it

- An experiment register. Every experiment is written down *before* it runs, with the result that would prove it wrong.
- “This is falsified if the harness doesn't beat the baseline outside seed variance.” That can fail. An opinion can't.
- It also fixes what can be fudged afterwards: which games are held back, how many seeds, which settings.

2:00-3:00

It has already earned it, twice

- EO's falsifier said “this is meaningless if the score is zero everywhere.” It nearly was, so the measurement switched to finer-grained ones before any model ran.
- This week's click fix was written as a test first: the click game must improve toward random, the keyboard game must not get worse. The click test passed. **The control test came back inconclusive, and it's recorded as inconclusive.**
- That last part is the point. Without writing the test first, I'd have kept the good half.

Part 2 – two pull requests to Genesis

Both found while using Genesis on this project, on Windows. Both are the same shape: something reported success while silently doing nothing.

PR #22 **Make Genesis work on Windows**

install.sh, tools/query.mjs • 3 commits

BUG 1

The query CLI printed nothing and exited 0. It compared its own module URL against `file://` plus the raw argument path. On Windows that builds `file://C:\kit\tools\query.mjs` while Node reports `file:///C:/kit/...`. The guard was never true, so `main()` never ran.

WHY IT MATTERS

`genesis query . impact PATH` is the blast-radius check the contract, the kickoff and all five recipes tell an agent to trust before editing shared code. Empty output there reads as “*nothing depends on this file*” — the one wrong answer that check must never give.

BUG 2

The installer reported success and installed a broken launcher. Git Bash without Developer Mode can't create symlinks, and `ln -s` copies instead of failing. The copy resolves its sibling imports next to itself, so every invocation died — including `--help`. Re-running the installer couldn't repair it, because its guard refused to replace the orphan.

FIX

Keep the symlink where symlinks work, *verify one was actually created*, and otherwise write a small launcher that runs the kit copy, so sibling imports resolve and `git pull` still updates it.

REVIEW CAUGHT

Two things in that launcher: the kit path was pasted into shell source unquoted, so a directory name containing `$` or a backtick could run as a command on every invocation; and ownership was checked by searching for a marker string, so any unrelated file mentioning it could be overwritten. Now the path is quoted properly and the whole three-line shape must match. Tested against twelve hostile paths.

EVIDENCE

Four existing tests already failed on Windows and now pass. They were the regression test all along.

PR #23 Resolve Python imports against declared source roots

`tools/graphizer.mjs` +37/-3 · tests +21

BUG	Projects using the standard <code>src/</code> layout had no first-party imports at all. Imports were resolved only against the repository root, so importing your own package by name never matched a file. Every one was filed as a third-party package from PyPI, and the link between files was never drawn.
WHY IT MATTERS	Same victim as PR #22: <code>impact</code> traverses imports. It reported <i>no dependents</i> for a file the tests import directly — and reported that as success.
FIX	Read the source root the project already declares — <code>setuptools</code>, <code>poetry</code> or <code>setup.cfg</code> — and fall back to the <code>src/</code> convention. Roots are tried <i>after</i> the repository root, so every link that resolved before resolves identically, and a candidate must already exist in the index to win. It can turn a wrong package label into a real file link; it can never invent one.
LIVE EXAMPLE	This ARC repo declares <code>where = ["src"]</code>. Genesis's index of it currently holds 38 PyPI-labelled entries, including <code>agents.agent</code>, <code>agents.swarm</code> and <code>e0_random_smoke</code> — our own code, filed as third-party packages.
FRAMING	This is the Python counterpart of the path-alias and workspace resolution JavaScript already had. Not a new feature: parity.

If you show one thing, show this

Run `genesis query . impact` on a file in a `src/` project. Before the fix it answers “nothing depends on this” and exits successfully. That's the whole argument for both pull requests in one command: the dangerous failure isn't a crash, it's a confident wrong answer.

Tie it back to the register: the reason we write down what would prove us wrong is the same reason a check must be able to fail. A green tick that can't go red isn't evidence.

Questions to expect

Are these Windows-only problems?

The CLI one is: the path comparison only breaks where paths aren't URL-shaped. The installer one shows up anywhere symlinks are unavailable. The Python one is platform-independent — any `src/` layout hits it.

How did you find them?

By using Genesis on a real project rather than reading it. Each surfaced as "this command did nothing and said it worked."

How do I review these?

Both are small. #22 is three commits across two files; #23 is one file plus tests. Start with the commit messages — each names the trigger, the user-visible consequence and the check that proves it.

Does the Python change risk breaking existing indexes?

Deliberately not. Source roots are tried after the repository root, so anything that resolved before resolves the same way, and a candidate must already be in the index — it can correct a label, never fabricate a dependency.

Why contribute upstream at all?

We rely on that blast-radius check daily. A fix that stays in my fork has to be re-applied on every update, and everyone else keeps the silent wrong answer.

Companion to *What Would Prove It Wrong* (artifact 920a5e75) and pull requests #22 and #23 on [ayush488-glitch/genesis-kit](#) , both open, from [Goutham-Eda/genesis-kit](#) . Compiled 12 September 2026.